PARECER TÉCNICO E PLANO DE AÇÃO

Validação de máquinas de estados finitos componíveis em Haskell

Análise das dificuldades relatadas pelo discente, soluções metodológicas e alternativas de código aberto

Base documental. Relatório *ProgressoParcial_v4.pdf* (38 páginas), especialmente seções 5.9 a 5.15, e comentários enviados pelo discente.

Objetivo deste parecer. Preservar o foco da pesquisa em segurança de tipos, composição de FSMs e verificação de comportamento, sem criar uma dependência desnecessária de simulação física de voo.

Síntese executiva

Manter o simulador puro em Haskell. Acrescentar uma especificação independente, testes baseados em modelos e, se viável, uma verificação de propriedades globais em TLA+/TLC. Auditar as traduções C/C++ antes de utilizá-las como evidência comparativa. A contribuição científica mais promissora está em delimitar o que a tipagem garante e o que depende de verificação comportamental.

Natureza: recomendação técnico-metodológica para orientação e preparação de artigo científico.

Data: 3 de novembro de 2026

1. Diagnóstico geral

O projeto estuda a formalização de FSMs em Haskell, utilizando ADTs, State Monad, composição monádica e a biblioteca CREM. O UAV é um **domínio ilustrativo**; o relatório deixa explícito que não há requisitos reais de segurança de voo. Logo, o objeto principal de validação é o comportamento discreto das máquinas e o alcance das garantias estáticas, e não a dinâmica física de um veículo aéreo.

Dificuldade relatada	Avaliação	Encaminhamento
Simulador desenvolvido em Haskell	Escolha metodológica defensável	Manter; validar por modelo externo e testes independentes
Eventos externos cruzam fronteira não tipada	Limitação real de integração, não perda automática da tipagem interna	Adaptador com decodificação, validação e testes de contrato
Necessidade de revisão técnica	Etapa importante para publicação	Revisão de propriedades, implementação e hipóteses
Pouca familiaridade com C/C++	Ameaça à validade da comparação	Auditoria das versões e configurações de compilação
Ausência de testes abertos encontrados	Lacuna do levantamento, não inexistência de alternativas	Reutilizar QuickCheck State Machine, Hedgehog e exemplos públicos
Desenvolvimento de simulador próprio	Não constitui problema científico por si só	Documentar semântica e confrontar com oráculo independente

Observação importante: o próprio relatório já inclui uma especificação em arquivo separado (seção 5.12) e um modelo de referência sem CREM (seção 5.13). Eles melhoram a validação, mas foram escritos pelo mesmo pesquisador, de modo que erros de interpretação podem estar presentes nos dois lados da comparação.

2. Resposta às dúvidas do discente

2.1. Simulador próprio ou simulador de voo externo?

A decisão de desenvolver uma simulação discreta em Haskell é adequada aos objetivos atuais. Incorporar um simulador físico de UAV aumentaria custo e complexidade sem necessariamente melhorar a avaliação da tipagem do CREM. A questão essencial é: **como verificar que o simulador executa a especificação pretendida?**

Solução: preservar o simulador atual, documentar entradas, estados, tempo discreto e políticas do ambiente, e compará-lo com uma segunda implementação ou uma especificação revisada independentemente.

2.2. A integração externa enfraquece a garantia do CREM?

O dado recebido de uma interface externa não tem, automaticamente, o tipo estático esperado em Haskell. Isso não elimina a garantia interna de uma ação CREM corretamente tipada. É necessário um **limite de confiança explícito**: recebimento de bytes ou mensagens, validação, conversão em ADTs, execução da FSM e comparação de saídas. A tipagem da ação restringe arestas; não prova que o evento escolhido conduz ao destino semanticamente correto.

2.3. C e C++ podem continuar na pesquisa?

Sim, como comparação **controlada**, após revisão independente. Evitar a conclusão ampla de que Haskell detecta defeitos que não podem ser detectados em C/C++; testes exaustivos, ferramentas estáticas e diferentes construções de tipos podem detectar categorias semelhantes. A contribuição verificável é a **antecipação** de certos diagnósticos para a compilação.

3. Projetos abertos recomendados

Os recursos abaixo têm propósitos diferentes. Nem todos são oráculos independentes: uma suíte ligada ao próprio CREM verifica sua implementação, enquanto uma especificação em outra linguagem pode reduzir a dependência do código testado. Links indicam os repositórios ou páginas públicas mencionados na análise anterior; versões e compatibilidade devem ser confirmadas antes da adoção.

CREM

Função: Exemplos e testes da biblioteca original; reprodução e regressão.

Uso proposto: Comparar topologias pequenas e rejeições de compilação; consultar o diretório de testes.

Cuidados: Não é independente da biblioteca em avaliação.

Abrir recurso

Motor

Função: Biblioteca alternativa de FSMs tipadas em Haskell.

Uso proposto: Comparar uma topologia equivalente e transições inválidas.

Cuidados: Abordagem e garantias não são necessariamente equivalentes às do CREM; verificar GHC.

Abrir recurso

QuickCheck State Machine

Função: Geração de comandos e testes orientados a modelos.

Uso proposto: Comparar CREM e modelo de referência; reduzir contraexemplos.

Cuidados: Exige modelo correto; usar uma versão mantida/compatível.

Abrir recurso

Hedgehog

Função: Testes por propriedades, geradores e shrinking.

Uso proposto: Explorar sequências maiores e cenários de divergência.

Cuidados: Amostragem não substitui prova geral.

Abrir recurso

TLA+ / TLC

Função: Especificação e exploração de estados alcançáveis.

Uso proposto: Verificar invariantes e propriedades temporais do modelo finito.

Cuidados: Verifica o modelo, não automaticamente a implementação Haskell.

Abrir recurso

PX4 Autopilot

Função: Projeto real de controle de UAV com documentação pública.

Uso proposto: Extrair requisitos para um segundo caso de estudo.

Cuidados: Não é oráculo direto da FSM simplificada; requisitos devem ser reconciliados.

Abrir recurso

4. Fronteira de integração tipada

Um adaptador permite utilizar um gerador externo de testes ou uma implementação de referência sem afirmar que os tipos Haskell se estendem ao software externo.

Gerador de entradas / oráculo externo

TLA+, teste por propriedades, C/C++ ou arquivo de cenários

↓

Adaptador de fronteira

Decodifica e valida dados externos; devolve Either Error AvEvent

↓

FSM em CREM

Recebe AvEvent e aplica ações com restrições de topologia

↓

Comparador de traços

Confronta estado final, saídas, passos intermediários e propriedades

Exemplo conceitual de validação de eventos:

```
decodeEvent :: String -> Either String AvEvent
decodeEvent "TAKEOFF"  = Right TakeoffSignal
decodeEvent "CRITICAL" = Right CriticalBattery
decodeEvent "LANDED"   = Right LandedSignal
decodeEvent "CUTOFF"   = Right SystemCutoff
decodeEvent _          = Left "Evento desconhecido"
```

A conversão correta para `AvEvent` restringe o formato e a enumeração dos eventos. Ela **não** demonstra que o evento determina a transição correta; isso precisa de especificação e de verificação própria.

5. Arquitetura de validação em três níveis

Nível 1 — Verificação estática

Preservar os testes de exaustividade do casamento de padrões, com avisos promovidos a erros, e a matriz **origem × destino**. O relatório apresenta 16 pares de vértices: 8 aceitos (incluindo identidades) e 8 rejeitados. A matriz **estado × evento** também tem 16 combinações no estudo, mas verifica outra propriedade: cobertura da função de transição. Não confundir as duas.

- Registrar testes que devem compilar e testes negativos que devem falhar, conferindo a causa do erro.
- Manter testes de mutação capazes de acusar arestas ilegais, destinos incorretos e eventos trocados.

Nível 2 — Especificação independente e model checking

Construir uma tabela de transições e de políticas revisada pelo orientador, derivada dos requisitos documentados em vez de copiar o código. Se viável, codificar uma versão compacta em TLA+ e verificar invariantes no TLC. Para propriedades temporais, explicitar pressupostos de progresso (por exemplo, existência de ticks futuros).

Propriedade candidata	Pergunta de verificação
Segurança da topologia	É impossível uma aresta direta InFlight → Shutdown?
Evento e destino	TakeoffSignal em Grounded leva ao destino especificado?
Política de bateria	O que ocorre se o sistema decola quando a bateria já está crítica?

Propriedade candidata	Pergunta de verificação
Composição	As saídas da planta e ambiente seguem o contrato para cada tick?
Progresso / terminação	A realimentação pode executar indefinidamente sem novos ticks?

Nível 3 — Testes baseados em modelos

Utilizar **QuickCheck State Machine ou Hedgehog** para gerar eventos, executar a implementação em CREM e confrontar os traços com o modelo. O valor adicional está na diversidade dos cenários, nos geradores reprodutíveis e no shrinking, não apenas em substituir a enumeração que o aluno já desenvolveu.

Traço de regressão prioritário: `Drain → Drain → TakeoffSignal → Idle → Idle`. Segundo o relatório, esse cenário expõe a diferença entre reação à *entrada* da bateria em estado crítico e reação à *persistência* nesse estado.

6. Auditoria da comparação com C e C++

O relatório já compara variantes com defeitos em C, C++ e Haskell/CREM. Como o discente relata pouca experiência em C/C++, o experimento precisa de revisão externa para evitar equivalências incorretas ou implementações injustamente fracas.

- Revisar estados, eventos, regras de domínio e resultados esperados das três linguagens.
- Reproduzir diagnósticos com opções de compilação publicadas; documentar versões e saídas completas.
- Tratar separadamente `enum` em C e `enum class` em C++.
- Considerar ao menos uma implementação moderna de C++ e distinguir compilação, testes e análise estática.
- Mensurar, quando possível, esforço de teste, manutenção e legibilidade dos diagnósticos.

Conclusão comparativa defensável

Nos experimentos apresentados, certas restrições de topologia são verificadas na compilação em CREM. Isso antecipa a detecção e dispensa testes específicos para essas arestas, mas não torna equivalentes a correção comportamental e a segurança de tipos. Um teste exaustivo pode encontrar erros sem que o compilador os rejeite.

7. Contribuição científica mais promissora

A contribuição mais original *em potencial* não é a simples implementação de uma FSM em CREM, mas a investigação dos limites de suas garantias locais diante de comportamentos globais da composição. A pesquisa deve verificar o estado da arte antes de alegar ineditismo.

Resultado relatado (seção 5.13)	Interpretação metodológica
87.381 cenários explorados com alfabeto reduzido até N = 8	Evidência delimitada ao domínio e à profundidade examinados
172 cenários terminam com bateria crítica, aeronave em voo e dois ticks ociosos	Contraexemplos do comportamento esperado sob a política original
Política que reage ao nível: 0 desses 172 casos	Correção observada para esse conjunto de cenários

Resultado relatado (seção 5.13)	Interpretação metodológica
5.841 cenários com estado final diferente entre políticas	Alteração relevante de comportamento; requer interpretação dos requisitos

Pergunta de pesquisa sugerida: Em que medida as garantias estáticas de transição oferecidas pelo CREM asseguram propriedades globais de máquinas de estados componíveis, e quais técnicas complementares são necessárias para verificar o comportamento?

Essa formulação desloca o foco de “o compilador aceita ou rejeita uma aresta?” para “quais propriedades são garantidas pela tipagem, quais exigem contratos de evento–destino e quais dependem de verificação temporal?”.

8. Plano de ação para a orientação

Etapa	Ação prioritária	Entregável verificável
1	Congelar versão e reproduzir os experimentos atuais	Log de build, lista de testes positivos e negativos, versão do GHC e dependências
2	Revisar requisitos e separar o oráculo do código	Tabela de transições e propriedades aprovada na orientação
3	Adotar QuickCheck State Machine ou Hedgehog	Geradores, casos mínimos e relatório de divergências
4	Criar especificação compacta em TLA+/TLC	Invariantes, estados alcançáveis e contraexemplos, se existirem
5	Auditar versões em C e C++	Matriz comparativa reproduzível e revisão de linguagem
6	Reescrever a contribuição do artigo	Questão de pesquisa, evidências, limites e ameaças à validade

Pauta recomendada para a próxima reunião

- Confirmar que o UAV permanece apenas um exemplo discreto, sem promessa de segurança de voo real.
- Revisar os 16 pares da topologia e as diferenças entre evento, aresta e propriedade temporal.
- Escolher um oráculo independente e a primeira ferramenta de testes a integrar.
- Definir os critérios de conclusão e quem revisará C/C++.
- Escolher o recorte científico do artigo e registrar limitações que não serão tratadas nesta fase.

9. Fontes e rastreabilidade

Documento do projeto: *ProgressoParcial_v4.pdf*, relatório de iniciação científica (2025), com foco nas seções 5.9 (segurança de tipos), 5.11 (simulador), 5.12 (especificação e matriz), 5.13 (cadeia da bateria), 5.14 (C/C++) e 5.15 (limitações).

Comentários do discente: justificativa do simulador em Haskell; limite de integração com fronteiras não tipadas; necessidade de revisão técnica; dúvidas sobre traduções em C/C++; dificuldade de encontrar exemplos e testes públicos.

Recursos públicos para avaliação e implementação

CREM — repositório da biblioteca: https://github.com/marcosh/crem

Motor — FSMs tipadas em Haskell: https://github.com/owickstrom/motor

QuickCheck State Machine: https://github.com/stevana/quickcheck-state-machine

Hedgehog — testes por propriedades: https://github.com/hedgehogqa/haskell-hedgehog

TLA+ / TLC: https://github.com/tlaplus/tlaplus

TLA+ Examples: https://github.com/tlaplus/Examples

PX4 Autopilot: https://github.com/PX4/PX4-Autopilot

PX4 — documentação de segurança: https://docs.px4.io/main/en/config/safety

---

Nota de escopo. Este parecer é uma análise do relatório do aluno, dos comentários encaminhados e de recursos públicos identificados anteriormente. Não inclui execução do repositório do aluno, inspeção do código-fonte completo, instalação de bibliotecas nem prova independente dos resultados numéricos. Os recursos sugeridos devem ter manutenção, licença, versão e compatibilidade confirmadas no momento da implementação.

Anexo — registro sintético das dificuldades do aluno

- Escolha de um simulador puramente em Haskell para manter a validação centrada na FSM, e não na física do voo.
- Preocupação com a perda de garantias estáticas quando eventos entram por bindings não tipados.
- Necessidade de revisão técnica do novo relatório parcial em orientação posterior.
- Insegurança para validar as traduções da mesma FSM em C e C++.
- Dificuldade em localizar testes e modelos de código aberto, motivando o desenvolvimento próprio.

Esses pontos foram reorganizados para análise; as formulações acima não são transcrições literais. A interpretação técnica e as recomendações estão nas seções 2 a 8.